Национальный исследовательский университет

«МЭИ»

Институт радиотехники и электроники

Кафедра радиотехнических систем

Аппаратура потребителей спутниковых радионавигационных систем

Задание на курсовой проект

по теме

«Расчет траектории движения спутника GPS по данным с демодулятора его сигнала»

ФИО преподавателя: Петухов Н.И.

Студент: Шипинский К.С.

Москва 2026

ЭТАП 1.

Цель курсового проекта– разработка модулей разбора навигационного сообщения GPS и расчета положения спутника, предназначенных для использования в составе навигационного приемника.

На первом этапе необходимо создать программу в среде C/ C++, выполняющую функции аналогичные модулю разбора навигационного сообщения. Листинг созданной части программы приведен в приложении.

На данном этапе реализован расчет всех целочисленных параметров спутника. В дальнейшем будет дописана программа для расчета остальных параметров (с плавающей точкой).

```
LNAV Ephemeris (slot = 14291) =
Crs     = 2.275000e+01
Dn     = 5.210052e-07     [deg/s]
M0     = -12.755586     [deg]
Cuc     = -1.557082e-06
e     = 1.636439e-02
Cus     = 6.183982e-06
sqrta    = 5.153701e+03
toe     = 288000
Cic     = -2.142042e-07
Omega0   = -147.307115     [deg]
Cis     = 1.266599e-07
i0     = 53.402052     [deg]
Crc     = 1.897500e+02
omega   = 82.744271     [deg]
OmegaDot = -4.756403e-07     [deg/s]
iDot    = -4.752011e-08     [deg/s]
Tgd     = -1.071021e-08
toc     = 288000
af2     = 0.000000e+00
af1     = 2.159915e-12
af0     = -2.265218e-04
WN     = 56
IODC    = 7
URA    = 0
Health = 0
IODE2   = 7
IODE3   = 7
codeL2 = 1
L2P     = 0
```

Рисунок 1 – Данные, полученные в ходе компиляции кода через командную строку.

Обработка BINR.bin и сравнение эфемерид

Для проверки корректности полученных эфемерид спутника PRN 24 был построен SkyPlot с помощью MATLAB-скрипта, реализующего алгоритм IS-GPS-200 и использующего координаты приёмника (Москва, МЭИ) и эфемериды из файла out.txt. Полученная траектория полностью совпадает с эталонной, формируемой сервисом Trimble GNSS Planning Online, что подтверждает верность расчётов, выполненных на Этапе 1.

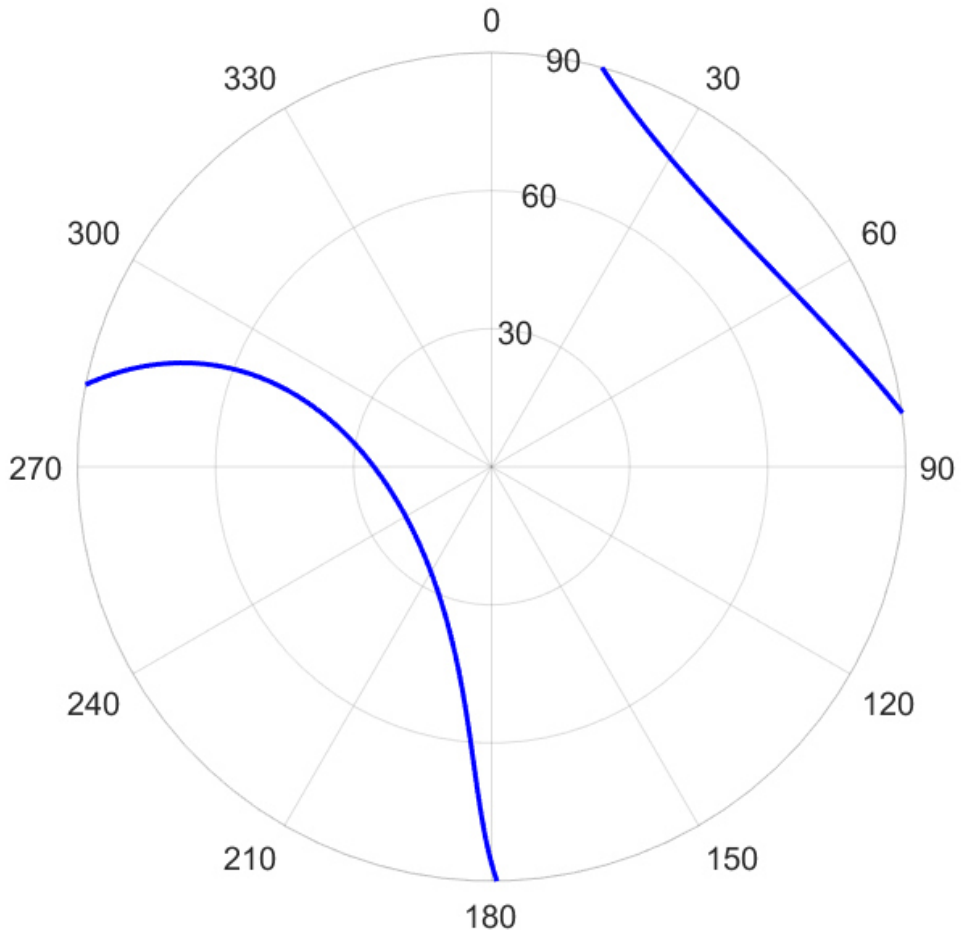

Рис. 1 – SkyPlot спутника GPS PRN 24 на дату наблюдений 3.03.2021, построенный по данным первого этапа. Сравнение с Trimble GNSS Planning Online подтверждает идентичность траектории.

Программа приведена на языке C++:

```
#define _CRT_SECURE_NO_WARNINGS
#include <cstdio>
#include <cstdint>
#include <cstdlib>
#include <cstring>
#include <cmath>

#define SUBFRAME_LEN 300

struct Ephemeris {
    uint32_t slot;
    double Crs, Dn, M0, Cuc, e, Cus, sqrtA, toe, Cic, Omega0, Cis, i0,
Crc, omega, OmegaDot, iDot;
    double Tgd, toc, af2, af1, af0;
    uint32_t WN;
    uint16_t iodc;
    uint8_t ura, health, IODE2, IODE3;
    uint8_t codeL2;
    bool L2P;
};

// ---------- Базовые операции с битами ----------
static uint64_t getbits(const char* bits, int start, int len) {
    uint64_t val = 0;
    for (int i = 0; i < len; i++)
        if (bits[start + i] == '1') val |= (1ULL << (len - 1 - i));
    return val;
}

static int64_t sbits(uint64_t raw, int bits) {
    int64_t sign = 1LL << (bits - 1);
    return (int64_t)((raw ^ sign) - sign);
}

// ---------- Поиск подкадров 1,2,3 для спутника 24 ----------
static int file2subFrames(const char* filename, char sf1[SUBFRAME_LEN+1],
                          char sf2[SUBFRAME_LEN+1], char
sf3[SUBFRAME_LEN+1], uint32_t* slot) {
    FILE* f = fopen(filename, "r");
    if (!f) return -1;
    char line[2048];
    bool have1 = false, have2 = false, have3 = false;

    while (fgets(line, sizeof(line), f)) {
        int prn = 0;
        char* p = strstr(line, "#");
        if (!p) continue;
        sscanf(p, "# %d", &prn);
        if (prn != 24) continue;   // только PRN 24

        char* bits = strstr(line, "10001011");
        if (!bits) continue;

        // Определяем subframe ID (последнее число перед битовой строкой)
        int subframeId = 0;
        char* before = bits - 1;
        while (before > line && (*before == ' ' || *before == '\t'))
before--;
        char* num_end = before + 1;
        while (num_end > line && num_end[-1] >= '0' && num_end[-1] <=
'9') num_end--;
```

```
        sscanf(num_end, "%d", &subframeId);
        int sfNum = (subframeId >> 3) & 0x7;
        if (sfNum < 1 || sfNum > 3) continue;

        // Слот (первое число после # prn)
        uint32_t cur_slot = 0;
        char* after = p + 1;
        while (*after == ' ' || *after == '\t') after++;
        sscanf(after, "%u", &cur_slot);

        // Сохраняем подкадр
        if (sfNum == 1 && !have1) {
            strncpy(sf1, bits, SUBFRAME_LEN);
            sf1[SUBFRAME_LEN] = '\0';
            *slot = cur_slot;
            have1 = true;
        } else if (sfNum == 2 && !have2) {
            strncpy(sf2, bits, SUBFRAME_LEN);
            sf2[SUBFRAME_LEN] = '\0';
            have2 = true;
        } else if (sfNum == 3 && !have3) {
            strncpy(sf3, bits, SUBFRAME_LEN);
            sf3[SUBFRAME_LEN] = '\0';
            have3 = true;
        }
        if (have1 && have2 && have3) break;
    }
    fclose(f);
    return (have1 && have2 && have3) ? 0 : 1;
}

// ---------- Декодирование эфемерид (IS-GPS-200 LNAV) ----------
static int subFrames2Eph(Ephemeris* ep, const char* sf1, const char* sf2,
const char* sf3) {
    // Subframe 1
    uint64_t w3 = getbits(sf1, 60, 30);
    ep->WN     = (w3 >> 20) & 0x3FF;
    ep->codeL2 = (w3 >> 18) & 0x3;
    ep->ura    = (w3 >> 14) & 0xF;
    ep->health = (w3 >> 8)  & 0x3F;
    uint8_t iodc_msb = (w3 >> 6) & 0x3;

    uint64_t w4 = getbits(sf1, 90, 30);
    ep->L2P = (w4 >> 29) & 1;

    uint64_t w7 = getbits(sf1, 180, 30);
    ep->Tgd = sbits((w7 >> 6) & 0xFF, 8) * pow(2, -31);

    uint64_t w8 = getbits(sf1, 210, 30);
    uint8_t iodc_lsb = (w8 >> 22) & 0xFF;
    ep->iodc = (iodc_msb << 8) | iodc_lsb;
    ep->toc = ((w8 >> 6) & 0xFFFF) * 16.0;

    uint64_t w9 = getbits(sf1, 240, 30);
    ep->af2 = sbits((w9 >> 22) & 0xFF, 8) * pow(2, -55);
    ep->af1 = sbits((w9 >> 6) & 0xFFFF, 16) * pow(2, -43);

    uint64_t w10 = getbits(sf1, 270, 30);
    ep->af0 = sbits((w10 >> 8) & 0x3FFFFF, 22) * pow(2, -31);

    // Subframe 2
```

```
    uint64_t w2_3 = getbits(sf2, 60, 30);
    ep->IODE2 = (w2_3 >> 22) & 0xFF;
    ep->Crs   = sbits((w2_3 >> 6) & 0xFFFF, 16) * pow(2, -5);

    uint64_t w2_4 = getbits(sf2, 90, 30);
    ep->Dn    = sbits((w2_4 >> 14) & 0xFFFF, 16) * pow(2, -43) * 180.0;
// перевод в град/с

    uint64_t w2_5 = getbits(sf2, 120, 30);
    uint64_t w2_6 = getbits(sf2, 150, 30);
    uint64_t w2_7 = getbits(sf2, 180, 30);

    // M0 (32 бита)
    uint32_t M0_raw =
        (((w2_4 >> 22) & 0xFF) << 24) |
        (((w2_5 >> 22) & 0xFF) << 16) |
        (((w2_6 >> 22) & 0xFF) << 8)  |
        (((w2_7 >> 22) & 0xFF));
    ep->M0 = sbits(M0_raw, 32) * pow(2, -31) * 180.0;

    // Cuc, Cus
    ep->Cuc = sbits((w2_5 >> 14) & 0xFFFF, 16) * pow(2, -29);
    ep->Cus = sbits((w2_6 >> 14) & 0xFFFF, 16) * pow(2, -29);

    // e (32 бита)
    uint64_t w2_8 = getbits(sf2, 210, 30);
    uint32_t e_raw =
        (((w2_7 >> 22) & 0xFF) << 24) |
        (((w2_7 >> 6)  & 0xFFFF) << 8) |
        (((w2_8 >> 22) & 0xFF));
    ep->e = sbits(e_raw, 32) * pow(2, -33);

    // sqrtA (32 бита)
    uint64_t w2_9 = getbits(sf2, 240, 30);
    uint32_t sqrtA_raw = ((w2_8 >> 6) & 0xFFFF) << 16 | ((w2_9 >> 6) &
0xFFFF);
    ep->sqrtA = sqrtA_raw * pow(2, -19);

    // toe (16 бит)
    uint64_t w2_10 = getbits(sf2, 270, 30);
    ep->toe = ((w2_10 >> 6) & 0xFFFF) * 16.0;

    // Subframe 3
    uint64_t w3_3 = getbits(sf3, 60, 30);
    ep->IODE3 = (w3_3 >> 22) & 0xFF;
    ep->Cic   = sbits((w3_3 >> 6) & 0xFFFF, 16) * pow(2, -29);

    uint64_t w3_4 = getbits(sf3, 90, 30);
    uint64_t w3_5 = getbits(sf3, 120, 30);
    uint64_t w3_6 = getbits(sf3, 150, 30);
    uint64_t w3_7 = getbits(sf3, 180, 30);
    uint64_t w3_8 = getbits(sf3, 210, 30);
    uint64_t w3_9 = getbits(sf3, 240, 30);
    uint64_t w3_10= getbits(sf3, 270, 30);

    // Omega0 (32 бита)
    uint32_t Omega0_raw =
        (((w3_4 >> 6) & 0xFFFFFF) << 8) |
        (((w3_5 >> 22) & 0xFF));
    ep->Omega0 = sbits(Omega0_raw, 32) * pow(2, -31) * 180.0;
```

```
    // Cis (16 бит)
    ep->Cis = sbits((w3_5 >> 6) & 0xFFFF, 16) * pow(2, -29);

    // i0 (32 бита)
    uint32_t i0_raw =
        (((w3_6 >> 6) & 0xFFFFFF) << 8) |
        (((w3_7 >> 22) & 0xFF));
    ep->i0 = sbits(i0_raw, 32) * pow(2, -31) * 180.0;

    // Crc (16 бит)
    ep->Crc = sbits((w3_7 >> 6) & 0xFFFF, 16) * pow(2, -5);

    // omega (32 бита)
    uint32_t omega_raw =
        (((w3_8 >> 6) & 0xFFFFFF) << 8) |
        (((w3_9 >> 22) & 0xFF));
    ep->omega = sbits(omega_raw, 32) * pow(2, -31) * 180.0;

    // OmegaDot (24 бита)
    uint32_t OmegaDot_raw =
        (((w3_9 >> 6) & 0xFFFF) << 8) |
        (((w3_10 >> 22) & 0xFF));
    ep->OmegaDot = sbits(OmegaDot_raw, 24) * pow(2, -43) * 180.0;

    // iDot (14 бит)
    ep->iDot = sbits((w3_10 >> 8) & 0x3FFF, 14) * pow(2, -43) * 180.0;

    return 0;
}

// ---------- Вывод в файл (формат Букалова) ----------
static void printEmp(const Ephemeris* ep) {
    FILE* out = fopen("out.txt", "w");
    if (!out) return;
    fprintf(out, "LNAV Ephemeris (slot = %u) =\n", ep->slot);
    fprintf(out, "Crs    = %.6e\n", ep->Crs);
    fprintf(out, "Dn    = %.6e    [deg/s]\n", ep->Dn);
    fprintf(out, "M0    = %.6f    [deg]\n", ep->M0);
    fprintf(out, "Cuc    = %.6e\n", ep->Cuc);
    fprintf(out, "e    = %.6e\n", ep->e);
    fprintf(out, "Cus    = %.6e\n", ep->Cus);
    fprintf(out, "sqrta   = %.6e\n", ep->sqrtA);
    fprintf(out, "toe    = %.0f\n", ep->toe);
    fprintf(out, "Cic    = %.6e\n", ep->Cic);
    fprintf(out, "Omega0  = %.6f    [deg]\n", ep->Omega0);
    fprintf(out, "Cis    = %.6e\n", ep->Cis);
    fprintf(out, "i0    = %.6f    [deg]\n", ep->i0);
    fprintf(out, "Crc    = %.6e\n", ep->Crc);
    fprintf(out, "omega  = %.6f    [deg]\n", ep->omega);
    fprintf(out, "OmegaDot = %.6e    [deg/s]\n", ep->OmegaDot);
    fprintf(out, "iDot   = %.6e    [deg/s]\n", ep->iDot);
    fprintf(out, "Tgd    = %.6e\n", ep->Tgd);
    fprintf(out, "toc    = %.0f\n", ep->toc);
    fprintf(out, "af2    = %.6e\n", ep->af2);
    fprintf(out, "af1    = %.6e\n", ep->af1);
    fprintf(out, "af0    = %.6e\n", ep->af0);
    fprintf(out, "WN    = %u\n", ep->WN);
    fprintf(out, "IODC   = %hu\n", ep->iodc);
    fprintf(out, "URA   = %hhu\n", ep->ura);
    fprintf(out, "Health = %hhu\n", ep->health);
    fprintf(out, "IODE2  = %hhu\n", ep->IODE2);
```

```
    fprintf(out, "IODE3  = %hhu\n", ep->IODE3);
    fprintf(out, "codeL2 = %hhu\n", ep->codeL2);
    fprintf(out, "L2P    = %hhu\n", ep->L2P);
    fclose(out);
}

int main() {
    char sf1[SUBFRAME_LEN+1] = {0};
    char sf2[SUBFRAME_LEN+1] = {0};
    char sf3[SUBFRAME_LEN+1] = {0};
    uint32_t slot = 0;

    if (file2subFrames("in.txt", sf1, sf2, sf3, &slot) != 0) {
        printf("Subframes for PRN 24 not found in in.txt!\n");
        return 1;
    }

    Ephemeris ep;
    memset(&ep, 0, sizeof(ep));
    ep.slot = slot;

    if (subFrames2Eph(&ep, sf1, sf2, sf3) != 0) {
        printf("Failed to decode ephemeris.\n");
        return 1;
    }

    printEmp(&ep);
    printf("Ephemeris successfully written to out.txt\n");
    return 0;
}
```